Labsheet 7

Que& Create a binary tree and Display its structure

```
# include <iostream>
using namespace std;

struct node {
    int data ;
    Node *left, *right;

};
int main ( ) {
    Noode*root = new node {1, NULL , NULL};
    root -> left = new node {2, NULL, NULL};
    root-> right = new node {3, NULL, NULL };

    cout << "root ; " << root -> data <<end1;
    Cout << "Left child ;" <<root -> left -> data<<en
    cout << "right child; " <<root-> right -> data ;

    return 0;

}
```

Output:

Root; 1

Left child : 2

right child: 3

Qu9 Inorder traversal of a binary tree

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *left, *right;
};

void inorder (Node *root) {
    if (root == NULL) return;
    inorder (root -> left);
    cout << root -> data << " ";
    inorder (root -> right);
}

int main () {
    Node *root = new node { 1, NULL, NULL };
    root -> left = new node {2, NULL , NULL };
    root -> right = new Node {3, NULL, NULL };
    inorder (root);
    return 0;
```

Output

2 1 3

Preorder Traversal of binary tree

```
#include<iostream>
using namespace std;
struct node {
    int data;
    Node *left, *right;
}
void preorder (Node *root) {
    if (root == NULL) return;
    cout << root -> data << " ";
    preorder (root -> left) ;
    Preorder (root -> right);
int main ( ) {
    Node *root = new node {1, NULL, NULL};
    root -> left = new node {2, NULL, NULL};
    root -> right = new node {3, NULL, NULL};
    Preorder (root);
    return 0;
}
```

output

123

Qu11 Postorder Traversal of a binary tree

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node * left, *right
};

void postorder (Node *root) {
    if (root == NULL) return;
    postorder (root -> left);
    postorder(root -> right);
    cout << root -> data << " ";
}

int main () {
    Node *root = new node {1, NULL, NULL};
    root -> left = new node {2, NULL, NULL};
    root -> right = new node {3, NULL, NULL};

    Postorder (root);
    return 0;
}
```

output:

2 3 1

Implement All three traversals

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *left, *right;
};

void in(Node* n) {
    if(!n) return;
    in(n->left);
    cout << n->data << " ";
    in(n->right);
}

void pre(node* n) {
    if(!n) return
    cout << n->data << " ";
    pre(n->left);
    pre(n->right);
}

void post(node* n) {
    if(!n) return;
    post(n->left);
    post(n->right);
    cout << n->data << " ";
}
```

```
int main() {
    Node* n1 = new Node{1, NULL, NULL};
    n1->left = new Node{2, NULL, NULL};
    n1->right = new Node{3, NULL, NULL};

    cout << "inorder; ";
    in(n1);
    cout << "\npreorder; ";
    Pre(n1);
    cout << "\npostorder: ";
    post(n1);
```

Output:

Inorder: 2 1 3

Preorder: 1 2 3

Postorder: 2 3 1

Create a binary search tree by inserting n elements

```
#include <iostream>
using namespace std;
Struct node {
    int data;
    Node *left, *right;
};
node* insert (node* cr, int x) {
    if (cr == NULL)
        return new Node {x, NULL, NULL};
    if (x < cr -> data)
        cr->left = insert (cr->left, x);
    else
        cr -> right = insert (cr->right, x);
    return cr;
}
void inorder (Node* cr) {
    if (cr == NULL) return;
    inorder (cr-> left);
    cout << cr-> data << " ";
    inorder (cr->right);
}
```

```
int main() {
    Node* root = NULL;
    int a[] = {50, 30, 70, 20, 40};

    for (int x; a)
        root = insert(root, x);

    inorder(root);
```

Output:

20 30 40 50 70

```
Key in a BST

ude <iostream>
namespace std;

ct Node {
int data;
Node *left, *right;


*insert (node* n, int x) {
 (!n) return new node {x, NULL, NULL};
 (x < n -> data)
   n -> left = insert (n-left, x);

n->right = insert (n=>right, x);
urn n;


rch (node* n, int x) {
(!n) return false;
(n -> data == x) return true;
 x < n -> data )

 return search (n-> left, x);
urn search (n->right, x) ;

in () {
de* n = NULL;
insert (n, 50);
insert (n, 30);
insert (n, 70);
```

```
int key = 30;
if (Search (or, key))
    cout << "key found";

else
    cout << key not found";
}
```

Output:

Key found

Delete a leaf node from a BST

```
#include <iostream>
using namespace std;

struct node {
    int data;
    Node *left, *right;
};

Node* insert (node* r, int x) {
    if (!r) return new Node {x, NULL, NULL};
    if (x < r->data) r->left = insert(r->left, x);
    else r->right = insert(r->right, x);
    return r;

Node* del (node* r, int x) {
    if (!r) return r;

if (x<r->data) r->left = del (r->left, x);
else if (x > r->data) r->right = del (r->right
, x);
else if (!r->left && !r->right) return NULL;

return r;

}
void inorder (Node* r) {
    if (!r) return;
    inorder(r->left);
    cout << r->data << " ";
    inorder (r->right);
```

```
}
int main() {
    Node* root = NULL;
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);

    root = del(root, 30);
    inorder(root);

}
```

Output:

50 70

Delete a node having one child from a BST

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *left, *right;
};

Node* insert(node* r, int x) {
    if (!r) return new node {x, NULL, NULL};
    if (x < r->data) r->left = insert(r->left, x);
    else r->right = insert(r->right, x);
    return r;
}

Node* del(node* r, int x) {
    if (!r) return r;
    if (x < r->data) r->left = del(r->left, x);
    else if (x > r->data) r->right = del(r->right,
    x);
    else {
        if (!r->left) return r->right;
        if (!r->right) return r->left;
```

```
    }
    return r;
}

void inorder (Node* r) {
    if (!r) return;
    inorder (r->left);
    cout << r->data << " ";
    inorder (r->right);
}

int main () {
    Node* r = NULL
    r = insert (r, 50);
    r = insert (r, 30);
    r = insert (r, 20);
    r = insert (r, 70);

    r = del (r, 30);
    inorder (r);
}
```

Output:

20 50 70

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *left, *right;
};

Node* insert (Node* n, int x) {
    if (!n) return new Node {x, NULL, NULL};
    if (x<n->data) n->left = insert (n->left,x);
    else n->right = insert (n->right, x);
    return n;
}

Node* minimum (Node* n) {
    while (n->left) n=n->left;
    return n;
}

Node* del (Node* n, int x) {
    if (!n) return n;
    if (x<n->data) n->left = del (n->left, x);
    else if (x>n->data) n->right = del (n->
    ,x);
    else {
```

```
if (!cr->left) return cr->right;
if (!cr->right) return cr->left;

Node* t = minimum(cr->right);

cr->data = t->data
cr->right = del(cr->right, t->data);

}
return cr;

}

void inorder(Node* cr) {
    if (!cr) return;
    inorder(cr->left);
    cout << cr->data << " ";
    inorder(cr->right);

}

int main() {
node* cr = NULL;

cr = insert(cr, 50);
cr = insert(cr, 30);
cr = insert(cr, 70);
cr = insert(cr, 60);
cr = insert(cr, 80);

cr = del(cr, 50);
inorder(cr);
```

culate the Height of a binary tree

```
include <iostream>
ig namespace std;

struct Node {
    int data;
    Node *left, *right;
};
int height (Node* n) {
    if (!n) return 0;

    int l = height (n->left);
    int r1 = height (n->right);

    return 1 + max (l, r1);
}
int main () {
    Node* n = new node {1, NULL, NULL};
    n-> left = new node {2, NULL, NULL};
    n-> right = new node {3, NULL, NULL};
    cout << "Height: " << height (n);
```

Output

Height: 2

```
Struct node {
    int data;
    Node *left, * right;
};
int count (Node* n) {
    if (!n) return 0;
    return 1+ count(n->left) +count (n->right);
}
int main() {
    node* n = new node {1. NULL, NULL};
    n-> left = new node {2. NULL, NULL};
    n-> right = new node {3. NULL, NULL};
    count << "total nodes: " << count (n);
```

Output

Total Nodes: 3

```
if (ch==1){
    cin>>n;
    root = insert(root,n);
}
else if (ch==2)
    inorder(root);
} while (ch!=3);

    return 0;
}
```

O/P :-

1. Insert 2. Inorder 3 Exit

1

50

1. Insert 2. Inorder 3. Exit

1

30

1. Insert 2. Inorder 3. Exit

2

30 50

1. Insert 2. Inorder 3. Exit

3